

Projekt Aplikacje Mobilne

Nawigacja Rowerowa

Stanisław Fiedler 160250

Michał Łatka 160263

Poznań 2026

1. Wstęp

Celem projektu było stworzenie aplikacji mobilnej wspomagającej nawigację rowerową na podstawie danych pochodzących z OpenStreetMap. Aplikacja umożliwia wyznaczanie tras rowerowych pomiędzy wskazanymi punktami, prezentowanie alternatywnych wariantów przejazdu oraz prowadzenie użytkownika po wybranej trasie w czasie rzeczywistym.

Jednym z głównych założeń projektu było preferowanie infrastruktury przyjaznej rowerzystom. Podczas wyznaczania tras uwzględniane i preferowane są ścieżki rowerowe i drogi lokalne, natomiast główne drogi nie są brane pod uwagę.

Aplikacja umożliwia również przechowywanie historii wyszukiwanych adresów. Zalogowani użytkownicy mogą zapisywać swoje wyszukiwania na koncie, dzięki czemu ostatnio używane adresy są automatycznie podpowiadane podczas wprowadzania kolejnych punktów trasy. Pozwala to na szybsze planowanie często wykorzystywanych przejazdów

2. Architektura aplikacji

Aplikacja została zaprojektowana w architekturze klient-serwer.

- klient: aplikacja Flutter (Dart)
- serwer routingowy: OSRM (Open Source Routing Machine)
- serwer danych i autoryzacji: PocketBase

Komunikacja z serwerem OSRM oraz PocketBase odbywa się za pomocą zapytań HTTP.

Aplikacja została napisana w języku Dart z wykorzystaniem frameworka Flutter. Flutter umożliwia tworzenie wieloplatformowych aplikacji mobilnych z jednego kodu źródłowego, co było głównym powodem jego wyboru. Interfejs użytkownika został przygotowany zgodnie z założeniami Material Design 3.

Do zarządzania stanem aplikacji wykorzystano bibliotekę Riverpod. Pozwala ona na wygodne współdzielenie danych pomiędzy ekranami oraz oddzielenie logiki biznesowej od warstwy prezentacji. W postaci providerów udostępniane są:

- aktualnie wybrane miasto,
- historia wyszukiwanych adresów,
- stan logowania użytkownika,
- wyszukiwanie trasy.

3. Budowa aplikacji

Aplikacja została podzielona na trzy główne ekrany odpowiadające kolejnym etapom planowania i realizacji podróży. Logowanie nie jest wymagane do korzystania z podstawowych funkcji programu. Użytkownik niezalogowany może wyszukiwać i przeglądać trasy, jednak historia adresów nie jest w jego przypadku zapisywana.

3.1. Wybór trasy

Pierwszym etapem jest określenie punktów, przez które ma przebiegać trasa. Użytkownik musi podać co najmniej dwa adresy: punkt początkowy oraz końcowy. Istnieje również możliwość dodawania punktów pośrednich za pomocą przycisku znajdującego się na pasku aplikacji. Podczas wprowadzania adresów dla zalogowanych użytkowników wyświetlane są podpowiedzi pochodzące z historii wcześniejszych wyszukiwań.

W menu bocznym dostępne są informacje o stanie logowania. Z tego poziomu użytkownik może:

- zalogować się,
- wylogować się,
- przeglądać zapisane adresy,
- usuwać wybrane rekordy z bazy danych.

3.2. Wybór wariantu trasy

Po wysłaniu zapytania do serwera OSRM aplikacja oczekuje na odpowiedź zawierającą dostępne warianty przejazdu. Zwrócone trasy są prezentowane na mapie w postaci nakładek graficznych. Jeżeli serwer zwróci kilka możliwych wariantów, użytkownik może porównać ich przebieg i wybrać najbardziej odpowiadającą mu opcję. Dla każdej trasy wyświetlane są podstawowe informacje:

- całkowita długość,
- przewidywany czas przejazdu,
- przebieg na mapie.

Komunikacja z serwerem odbywa się asynchronicznie, dzięki czemu interfejs pozostaje responsywny nawet podczas oczekiwania na odpowiedź.

3.3. Nawigacja

Po wybraniu konkretnej trasy użytkownik przechodzi do widoku nawigacji. Mapa zostaje automatycznie przybliżona do bieżącej lokalizacji użytkownika, która jest stale aktualizowana na 2 podstawie danych GPS urządzenia. Pozycja użytkownika jest utrzymywana na środku ekranu, co ułatwia śledzenie postępu podróży. Dzięki temu aplikacja może pełnić funkcję prostego systemu nawigacyjnego podczas jazdy rowerem.

4. Zaimplementowane Funkcjonalności

4.1. Zdalna baza danych

Zdalna baza danych obsługiwana jest przez PocketBase uruchomiony jako docker na serwerze. Reguły dostępu skonfigurowane w PocketBase uniemożliwiają odczyt lub modyfikację danych należących do innych użytkowników

```
final pbProvider = Provider<PocketBase>((ref) {
  final prefs = ref.watch(sharedPreferencesProvider);
  const storageKey = 'pb_auth';
  final cachedAuth = prefs.getString(storageKey);

  final authStore = AsyncAuthStore(
    initial: cachedAuth,
    save: (String data) async {
      await prefs.setString(storageKey, data);
    },
    clear: () async {
      await prefs.remove(storageKey);
    },
  ),
```

```

);

return PocketBase('http://192.168.1.102:8090', authStore: authStore);
});

Future<List<String>> _fetchFromDatabase(String userId) async {
  final pb = ref.read(pbProvider);
  try {
    final records = await pb.collection('search_history').getList(
      filter: 'user = "$userId"',
      sort: '-created',
    );
    return records.items.map((item) => item.getStringValue('query')).toList();
  } catch (_) {
    return [];
  }
}

```

4.2. Interfejs użytkownika w całości oparty na Material Design

Interfejs aplikacji został wykonany w technologii Flutter z wykorzystaniem Material Design 3. Fragment widgetu:

```

return Scaffold(
  appBar: AppBar(title: const Text('Route Result')),
  floatingActionButton: routes.isNotEmpty
    ? FloatingActionButton.extended(
        onPressed: () {
          Navigator.of(context).push(
            MaterialPageRoute(
              builder: (context) => ActiveNavigationScreen(
                selectedRoute: routes[selectedRouteIndex],
              ),
            ),
          );
        },
        label: const Text('Start Route'),
        icon: const Icon(Icons.navigation),
        backgroundColor: Colors.blue,
      )
    : null,
  body: SafeArea(
    child: LayoutBuilder(
      builder: (BuildContext cntx, BoxConstraints constraints) {
        return Column(
          children: [
            LimitedBox(
              maxHeight: constraints.maxHeight / 3,
              child: ListView.builder(
                shrinkWrap: true,
                itemCount: widget.waypoints.length,
                itemBuilder: (BuildContext cntx, int index) {
                  return getLabel(index);
                },
              ),
            ),
          ],
        ),
      ),
    ),
  ),

```

4.3. Zastosowanie bezpiecznych mechanizmów identyfikacji i uwierzytelniania

Jako system autoryzacji wykorzystano PocketBase. Każdy użytkownik posiada własny zestaw zapisanych adresów. Reguły dostępu skonfigurowane w PocketBase uniemożliwiają odczyt lub modyfikację danych należących do innych użytkowników.

```
class AuthNotifier extends Notifier<AuthState> {
  @override
  AuthState build() {
    final pb = ref.watch(pbProvider);
    return AuthState(isAuthenticated: pb.authStore.isValid);
  }

  Future<void> login(String email, String password) async {
    state = AuthState(isLoading: true);
    final pb = ref.read(pbProvider);

    try {
      await pb.collection('users').authWithPassword(email, password);
      state = AuthState(isAuthenticated: true);
    } catch (e) {
      state = AuthState(
        isLoading: false,
        errorMessage: e.toString().contains('400')
          ? 'Invalid email or password'
          : 'Connection error',
      );
    }
  }
}
```

4.4. Lokalizacja i zastosowanie serwisów mapowych

W aplikacji wyświetlana jest mapa OSM z warstwą rowerową, na nią nakładana jest lokalizacja z GPS.

```
OSMFlutter(
  controller: controller,
  onMapIsReady: (isReady) async {
    if (!isReady) return;

    await controller.drawRoadManually(
      widget.selectedRoute.points,
      const RoadOption(
        roadColor: Colors.blue,
        roadWidth: 12,
      ),
    );

    await controller.moveTo(widget.selectedRoute.points.first);

    if (!_isTracking) {
      _isTracking = true;
      await _startLocationTracking();
    }

    await controller.currentLocation();
  },
)
```

```
osmOption: const OSMOption(  
    userTrackingOption: UserTrackingOption(  
        enableTracking: true,  
        unFollowUser: false,  
    ),  
    showDefaultInfoWindow: false,  
    showContributorBadgeForOSM: false,  
    zoomOption: ZoomOption(  
        initZoom: 18,  
        minZoomLevel: 12,  
        maxZoomLevel: 19,  
    ),  
    roadConfiguration: RoadOption(  
        roadColor: Colors.transparent,  
    ),  
),  
)
```

4.5. Serwisy zewnętrzne

Do wyznaczania tras wykorzystano Open Source Routing Machine (OSRM) uruchomiony na serwerze w kontenerze Docker.

Aplikacja komunikuje się z nim za pomocą interfejsu HTTP API. Użytkownik podaje adresy początkowe i końcowe (ewentualnie pośrednie), które następnie są zamieniane na współrzędne geograficzne z wykorzystaniem pakietu geocoding. Po otrzymaniu współrzędnych aplikacja wysyła zapytanie do serwera OSRM, który zwraca jedną lub więcej tras.

Do wyznaczania tras wykorzystywany jest profil bike. Dzięki temu preferowane są ścieżki rowerowe oraz drogi odpowiednie dla rowerzystów.

```
class RouteService {
    static Future<List<RouteOption>> fetchRoute({
        required List<String> waypoints,
        required City city,
    }) async {
        final geoPoints = await geocodeWaypoints(waypoints, city);

        final coords = geoPoints.map((p) => '${p.lon},${p.lat}').join(';');
        final url = 'http://192.168.1.102:5000/route/v1/bike/$coords'
            '?steps=true&overview=full&geometries=geojson'
            '&alternatives=true';

        final response = await http.get(Uri.parse(url));
        final data = jsonDecode(response.body);
        final routes = data['routes'] as List;

        final options = routes.asMap().entries.map((entry) {
            final index = entry.key;
            final route = entry.value;

            final coordinates = route['geometry']['coordinates'] as List;
            final points = coordinates.map<GeoPoint>((c) {
                return GeoPoint(latitude: c[1], longitude: c[0]);
            }).toList();
        });
    }
}
```

```

final double distance = (route['distance'] as num).toDouble();
final double duration = (route['duration'] as num).toDouble();

return RouteOption(
  name: index == 0 ? 'Fast Route' : 'Alternative $index',
  points: points,
  distance: distance,
  duration: duration,
);
}).toList();

return options;
}
}

```

4.6. Asynchroniczność i komunikacja sieciowa

Funcje sieciowe aplikacji są zaimplementowane asynchronicznie co widać np. w punktach Baza danych 4.1 oraz Serwisy zewnętrzne 4.5.

5. Ekrany aplikacji

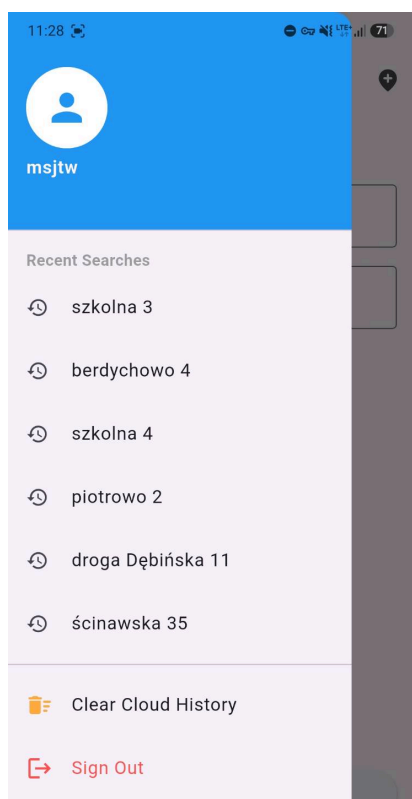


Figure 1: Ekran użytkownika

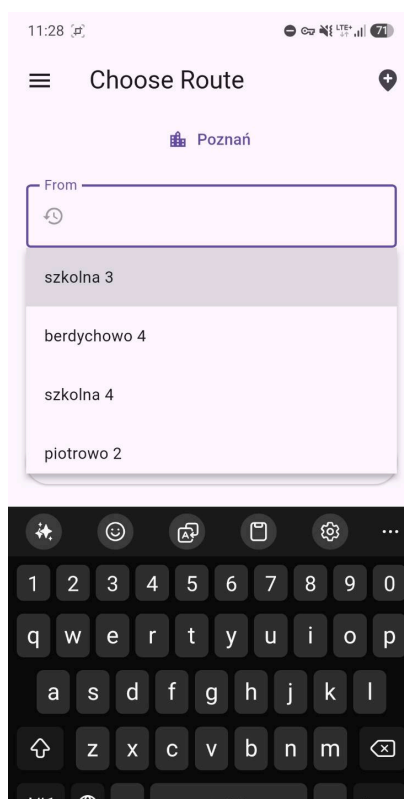


Figure 2: Wyszukiwanie trasy

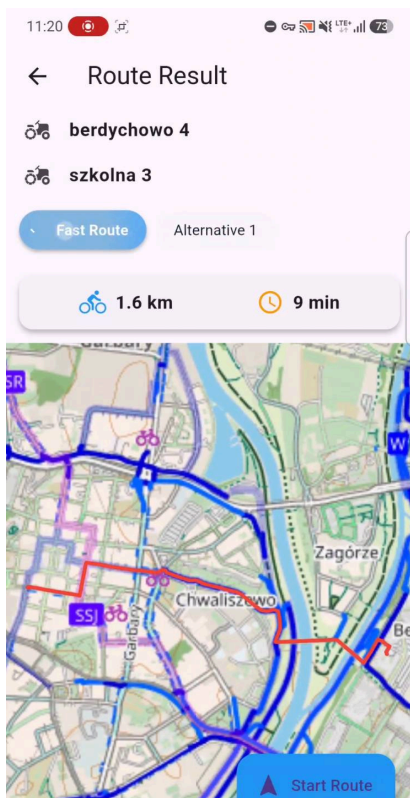


Figure 3: Wybór trasy

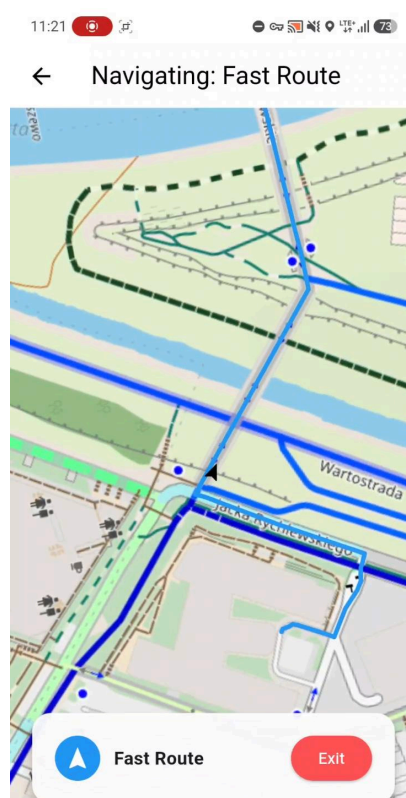


Figure 4: Nawigacja

6. Podsumowanie

W ramach projektu powstała aplikacja mobilna do planowania tras rowerowych. Program umożliwia wyszukiwanie tras pomiędzy wskazanymi punktami, wybór spośród kilku wariantów przejazdu oraz nawigowanie użytkownika przy wykorzystaniu danych GPS. Połączenie technologii Flutter, Riverpod, OSRM oraz PocketBase pozwoliło stworzyć kompletne rozwiązanie obejmujące zarówno interfejs użytkownika, logikę biznesową, system uwierzytelniania, jak i usługę wyznaczania tras.

Największym ograniczeniem zastosowanego rozwiązania jest konieczność lokalnego przechowywania map przez serwer OSRM. Ze względu na ograniczone zasoby serwera do projektu wykorzystano jedynie dane obejmujące województwo wielkopolskie. Rozszerzenie działania na teren całej Polski lub Europy wymagałoby pobrania znacznie większych zbiorów danych OpenStreetMap oraz ich dodatkowego przetworzenia.